EEG Analysis Pipeline — Two-Group Cognitive Performance Study

A complete EEG preprocessing and ERP analysis pipeline for Stroop and N-back tasks

> Two-group cognitive performance study (e.g., Treatment vs Control)
> EEG system: BrainVision actiCHamp, 32 channels, 500 Hz
> Analysis: MNE-Python | Python 3.10+

Table of Contents

---

1. Introduction

What is EEG?

Electroencephalography (EEG) measures electrical activity produced by the brain through electrodes placed on the scalp. When neurons fire, they produce tiny electrical signals that can be detected at the scalp surface. EEG records these signals continuously over time, typically at hundreds of samples per second.

What is an ERP?

An Event-Related Potential (ERP) is the brain's electrical response to a specific event — for example, seeing a word on screen, hearing a sound, or making a decision. To measure an ERP, we time-lock the EEG signal to many repetitions of the same event and average them together. Random background brain activity cancels out in the average, leaving behind the consistent response to the event of interest.

What does this pipeline do?

This pipeline takes raw EEG recordings and E-Prime behavioural data from a study comparing two experimental groups (e.g., treatment vs. control) on two cognitive tasks:

- **Stroop task** — participants name the ink colour of colour words (e.g. the word RED printed in blue ink). Incongruent trials (word and colour do not match) require conflict resolution.
- **N-back task** — participants monitor a sequence of letters and respond when the current letter matches the one shown N positions back. Tests working memory.

The pipeline processes the raw EEG data through a series of steps to produce publication-ready ERP waveform figures, component amplitude and latency measurements, and between-group statistical comparisons.

Who is this for?

This README assumes you: - Are familiar with basic computing (opening terminals, navigating folders) - Have some understanding of what EEG is - Are new to Python or to this specific pipeline

No prior Python programming experience is required to run the pipeline — you only need to type commands as shown.

1.5 Assumptions & Prerequisites

> [!IMPORTANT] **What this pipeline assumes about your data and experiment:**
>
> - **Hardware:** Designed specifically for 32-channel EEG setups (e.g., BrainVision actiCHamp) at 500 Hz sampling rate. It expects standard 10-20 system electrode locations including `FC1`, `FC2`, `Fz`, `Cz`, `Pz`.
> - **Task Designs:**

    - *Stroop task:* Expects incongruent and congruent trials.
    - *N-back task:* Expects target and non-target trials, across varying loads.
- **Trigger Codes:** It expects E-Prime hardware trigger markers embedded in the `.vmrk` file to time-lock the epochs. For example, Stroop expects S3 for stimulus, S5 for incongruent correct. If your triggers differ, you must modify `04_epochs.py` before running.

---

2. Installing Python

Python is a free, open-source programming language. All scripts in this pipeline are written in Python 3.

How to open a terminal

Before installing or running anything, you need to open a terminal — the text-based interface where you type commands.

Windows — Command Prompt: 1. Press the **Windows key** on your keyboard 2. Type `cmd` in the search bar 3. Click **Command Prompt** in the results 4. A black window will open with a prompt like `C:\Users\yourname>`

Windows — PowerShell (alternative, also works): 1. Press the **Windows key** 2. Type `powershell` 3. Click **Windows PowerShell**

Mac — Terminal: 1. Press **Command (⌘) + Space** to open Spotlight search 2. Type `terminal` 3. Press **Enter** or click **Terminal** in the results 4. A window will open with a prompt like `yourname@MacBook ~ %`

> **Keep this window open** — all commands in this guide are typed here.

Installing Python on Windows

1. Go to https://www.python.org/downloads/
2. Click **Download Python 3.11.x** (or the latest 3.10+ version shown)
3. Run the downloaded installer file
4. **Critical:** On the first installer screen, tick the checkbox **"Add Python to PATH"** before clicking anything else
5. Click **Install Now**
6. When complete, close and reopen Command Prompt, then verify:

Windows:

```
python --version
```

You should see something like `Python 3.11.4`

Installing Python on Mac

Mac comes with an older Python version built in. Install the current version:

1. Go to https://www.python.org/downloads/
2. Download the macOS installer (`.pkg` file)
3. Run it and follow the steps
4. Open Terminal and verify:

Mac:

```
python3 --version
```

You should see `Python 3.11.x` or similar.

> **Important note for Mac users:** On Mac, the command is `python3` (not `python`). This distinction applies to every command in this guide — wherever you see `python`, Mac users should type `python3` instead. This will be noted explicitly for every command example.

Virtual Environment (recommended)

A virtual environment is an isolated space where you install packages for this project only, without affecting other Python projects on your computer. Think of it as a clean room dedicated to this pipeline like a mirror dimension used in dr strange movies, whatever happens here does not affect the real world and you can always exit it by typing 'deactivat.

Windows — create and activate:

```
cd C:\Users\yourname\eeg_analysis
python -m venv .venv
.venv\Scripts\activate
```

Mac — create and activate:

```
cd /Users/yourname/eeg_analysis
python3 -m venv .venv
source .venv/bin/activate
```

When activated, you will see `(.venv)` at the start of your command line. All package installations go into the virtual environment only.

To deactivate when you are done:

Windows and Mac:

```
deactivate
```

---

3. Required Libraries

A library (also called a package or module) is a collection of pre-written code that extends Python's capabilities. This pipeline uses the following libraries:

Library	What it does
MNE	The main EEG analysis library — loads raw data, filters, runs ICA, creates epochs, computes ERPs, plots topomaps
mne-icalabel	Automated ICA component classification using the ICLabel neural network — labels components as brain, eye blink, muscle, heartbeat, or noise. Used in `03_ica.py`. **Separate from MNE and must be installed explicitly.**
onnxruntime	Inference backend required by `mne-icalabel` to run the ICLabel model. Lightweight (~10 MB). **Must be installed alongside `mne-icalabel` — without it, `03_ica.py` will crash with an `ImportError`.**
NumPy	Numerical computing — handles arrays, matrix operations, mathematical functions
SciPy	Scientific computing — statistical tests (Welch t-test, Mann-Whitney U)

Library	What it does
statsmodels	Advanced statistical methods — provides the Benjamini-Hochberg FDR correction (`multipletests`) applied to all p-values in `07_statistics.py`. **Separate from SciPy and must be installed explicitly.**
pandas	Data tables — loads and saves CSV files, organises behavioural data
matplotlib	Plotting — generates all ERP waveform figures and bar charts

> **Important:** `mne-icalabel` is not included in MNE itself. It is a separate package maintained independently. If you install only `mne`, the ICA script (`03_ica.py`) will fail with a `ModuleNotFoundError` when it reaches the automated classification step.

Recommended — install everything from `requirements.txt`

The project includes a `requirements.txt` file in the project root that lists every dependency. With your virtual environment activated, run this single command:

Windows:

```
python -m pip install -r requirements.txt
```

Mac:

```
python3 -m pip install -r requirements.txt
```

This installs all libraries in one step and ensures nothing is missed. You only need to do this once per environment.

Alternative — install libraries individually

If you prefer to install packages one by one, run the following with your virtual environment activated:

Windows:

```
python -m pip install mne mne-icalabel onnxruntime numpy scipy statsmodels pand
```

Mac:

```
python3 -m pip install mne mne-icalabel onnxruntime numpy scipy statsmodels pan
```

> **Note:** `mne-icalabel`, `onnxruntime`, and `statsmodels` must all be installed individually — none of them are pulled in automatically by the others.

Verifying the installation

To confirm all key packages installed correctly:

Windows:

```
python -c "import mne; import mne_icalabel; import statsmodels; print('MNE:', m
```

Mac:

```
python3 -c "import mne; import mne_icalabel; import statsmodels; print('MNE:',
```

You should see version numbers printed with no errors. If you see a `ModuleNotFoundError` for any package, re-run the install command above with your virtual environment activated.

> **What is pip?** `pip` is Python's package manager — it downloads and installs libraries from the internet automatically. It is installed alongside Python.

> **First-run note for `03_ica.py`:** The first time `mne-icalabel` runs, it will automatically download its pre-trained ICLabel model weights from the internet (~a few MB via `pooch`). This requires an internet connection on first use only. The weights are cached locally afterwards. `onnxruntime` is what actually executes this model — it is not optional.

---

4. Setting Up the Project

Step 1 — Download the pipeline scripts

Download or clone this repository to your computer. Place the folder wherever you want your project to live, for example:

- **Windows:** `C:\Users\yourname\eeg_analysis\`
- **Mac:** `/Users/yourname/eeg_analysis/`

Step 2 — Navigate to your project folder

Before running any script, you must navigate your terminal to the project folder. This tells the terminal where to look for files.

Windows:

```
cd C:\Users\yourname\eeg_analysis
```

Mac:

```
cd /Users/yourname/eeg_analysis
```

> **What is `cd`?** It stands for "change directory" — it moves you into a folder. Replace the path shown with the actual location of your project folder on your computer.

To confirm you are in the right place, list the files:

Windows:

```
dir
```

Mac:

```
ls
```

You should see `scripts/`, `data/`, `00_setup_folders.py`, and the runner scripts listed.

Step 3 — Run the setup script

Windows:

```
python 00_setup_folders.py
```

Mac:

```
python3 00_setup_folders.py
```

The setup tool will ask what you want to do:

```
[1] Create new project folder structure at a location I choose
[2] Check an existing project folder structure
[3] Show file type instructions for all folders
[4] Exit
```

Choose **[1]** for a new project. It will: - Create all required input folders - Copy pipeline scripts into the correct locations if found alongside the setup script - Generate a template `participants.csv`

Step 4 — Edit participants.csv

Open `data/participants.csv` in any text editor or Excel. Fill in the correct details for each participant:

```
participant_id,name,group,age,sex
P01,John Doe,Group2,24,M
P02,Jane Smith,control,22,F
...
```

The `group` column must be exactly as you named them during setup (e.g., `Group1` or `Group2`).

Step 5 — Place your data files

See **Section 6** for exactly which files go where.

Step 6 — Verify everything is in place

Run the setup tool again and choose **[2] Check**:

Windows:

```
python 00_setup_folders.py
```

Mac:

```
python3 00_setup_folders.py
```

It will scan all folders and report what is present, what is missing, and whether the pipeline is ready to run.

---

5. Folder Structure

```
eeg_analysis/                          ← project root
│
├── 00_setup_folders.py                ← setup and validation tool
├── run_pipeline.py                    ← runs full pipeline (N-back or Stroop)
├── run_pipeline.sh                    ← runs full pipeline (bash wrapper)
│
├── scripts/                           ← all analysis scripts
│   ├── 01_parse_eprime.py
│   ├── 02_import_filter_eeg.py
│   ├── 03_ica.py
│   ├── 04_epochs.py
│   ├── 05_erp.py
│   ├── 06_group_erp.py
│   ├── 07_statistics.py
│   ├── 08_EEG_variation.py
│   └── 09_behavioural.py
│
├── data/                              ← all input data
│   ├── participants.csv               ← participant list and group assignments
│   ├── raw/                           ← raw EEG files (.vhdr .vmrk .eeg)
│   ├── processed/                     ← intermediate EEG files (auto-created)
│   └── behavioural/
│       ├── stroop/                    ← E-Prime Stroop .txt files
│       ├── nback/                     ← E-Prime N-back .txt files
│       └── edat_backup/               ← original .edat2 files (backup only)
│
└── output/                            ← all results (auto-created)
    ├── epochs/                        ← epoch rejection reports
    ├── erp/                           ← individual participant ERP plots and C
    ├── group/                         ← group-level ERP plots and CSVs
    └── stats/                         ← statistical results CSVs
```

> **Note:** The `data/processed/` and all `output/` subfolders are created automatically when you run the pipeline scripts. You do not need to create them manually.

---

6. Input Files — What You Need

6.1 Raw EEG files — `data/raw/`

EEG data recorded with BrainVision actiCHamp is stored in three files per recording session. All three must be present.

File extension	What it contains
`.vhdr`	**Header file** — text file containing recording parameters: number of channels, sampling rate, channel names, and the filenames of the matching `.vmrk` and `.eeg` files
`.vmrk`	**Marker file** — text file containing trigger codes sent by E-Prime during the experiment: when each stimulus appeared, what the participant's response was
`.eeg`	**Binary data file** — the actual EEG signal, one value per channel per time point. Usually the largest file (hundreds of MB)

Naming rule — extremely important:

All three files must share exactly the same base name. For example:

```
JD_Stroop_22_05_2026.vhdr
JD_Stroop_22_05_2026.vmrk
JD_Stroop_22_05_2026.eeg
```

> ⚠️ **Golden rule: Never rename `.vhdr`, `.vmrk`, or `.eeg` files after recording.** The `.vhdr` file contains internal references to the other two files by their exact filenames. If you rename any file, these internal references break and the data cannot be loaded.

One set of three files per participant per task — 16 sets (48 files total) for 8 participants and 2 tasks.

6.2 E-Prime behavioural files — `data/behavioural/`

File format: Tab-separated text, UTF-16 encoded (`.txt`)

> ℹ️ **Do I already have a `.txt` file?** E-Prime can be configured to automatically save a `.txt` file alongside the `.edat2` binary file during the experiment session. Check the folder where your `.edat2` files are stored — a matching `.txt` may already be there.

If no `.txt` file exists — export from E-DataAid:

1. Open **E-DataAid** (installed with E-Prime)
2. Go to **File → Open** and select the participant's `.edat2` file
3. Go to **File → Export → Tab-delimited text**
4. Set: Format = Tab-delimited text, Encoding = **Unicode (UTF-16)**, Include = All variables
5. Save as `P01_stroop.txt` or `P01_nback.txt`

File naming:

```
data/behavioural/stroop/  →  P01_stroop.txt ... P08_stroop.txt
data/behavioural/nback/   →  P01_nback.txt  ... P08_nback.txt
```

Use underscores only — spaces in filenames cause errors. Keep `.edat2` files in `data/behavioural/edat_backup/`.

6.3 participants.csv — `data/`

Column	Example
`participant_id`	P01
`name`	John Doe
`group`	exactly as named during setup
`age`	24
`sex`	M or F

6.4 outliers.csv — `data/` (Optional)

Create an `outliers.csv` file in the `data/` directory to explicitly flag specific participants to be excluded from certain conditions during the statistical analysis.

Column	Example	Description
`participant_id`	P01	The participant to exclude
`task`	stroop	The task (stroop or nback)
`condition`	incongruent/correct	The specific condition to exclude them from (or `all` to exclude from all conditions)

---

7. Pipeline Overview

```
Raw EEG + E-Prime data
        │
        ▼
[01] Parse N-back behavioural    ──► data/behavioural/nback/*_clean.csv
[02] Parse Stroop behavioural    ──► data/behavioural/stroop/*_clean.csv
        │
        ▼
[03] Import and filter EEG       ──► data/processed/*_filtered_raw.fif
        │
        ▼
[04] ICA artefact removal        ──► data/processed/*_ica_raw.fif
        │   ↑ HUMAN REVIEW REQUIRED
        ▼
[05] Epoching                    ──► data/processed/*_epo.fif
        │
        ├── Stroop ──► [06_stroop] ──► [07_stroop] ──► [08_stroop]
        └── N-back ──► [06_nback]  ──► [07_nback]  ──► [08_nback]
```

---

8. Script-by-Script Guide

`00_setup_folders.py` — Project Setup

What it does: Creates the project folder structure, copies pipeline scripts if found alongside the setup file, validates all required input files are present.

When to run: Once at the start of a new project, or any time you want to check the project status.

Windows:

```
python 00_setup_folders.py
```

Mac:

```
python3 00_setup_folders.py
```

`01_parse_eprime.py` — Parse Behavioural Data

What it does: Reads the raw E-Prime `.txt` files for the specified task, extracts trial-level data (reaction times, accuracy, trial type), separates practice from experimental trials, joins participant group information from `participants.csv`, and saves a clean CSV per participant.

When to run: Once per study per task, before running script 04.

Windows:

```
python scripts/01_parse_eprime.py --task nback
python scripts/01_parse_eprime.py --task stroop
```

Mac:

```
python3 scripts/01_parse_eprime.py --task nback
python3 scripts/01_parse_eprime.py --task stroop
```

Reads: `data/behavioural/<task>/P0X_<task>.txt` + `data/participants.csv`

Produces: `data/behavioural/<task>/P0X_<task>_clean.csv` — one row per trial with participant ID, group, and task-specific extracted features.

`02_import_filter_eeg.py` — Import and Filter EEG

What it does: Loads the raw BrainVision EEG file, sets electrode positions, re-adds the Cz reference electrode, applies a bandpass filter (0.1–40 Hz) and notch filter (50 Hz, 100 Hz).

When to run: Once per participant per task. The vhdr filename is the actual filename of your recording file.

Windows:

```
python scripts/02_import_filter_eeg.py P01 stroop JD_Stroop_22_05_2026.vhdr
python scripts/02_import_filter_eeg.py P01 nback  JD_Nback_22_05_2026.vhdr
```

Mac:

```
python3 scripts/02_import_filter_eeg.py P01 stroop JD_Stroop_22_05_2026.vhdr
python3 scripts/02_import_filter_eeg.py P01 nback  JD_Nback_22_05_2026.vhdr
```

Filtering applied:

Filter	Purpose
0.1 Hz highpass	Removes slow baseline drift and sweat artefacts
40 Hz lowpass	Removes high-frequency muscle noise
50 Hz notch	Removes UK mains electrical interference
100 Hz notch	Removes first harmonic of mains frequency

Produces: `data/processed/P01_stroop_filtered_raw.fif`

> **What is a `.fif` file?** FIF is MNE's native file format. It stores the EEG signal, channel information, electrode positions, and event markers in a single compressed file.

`03_ica.py` — ICA Artefact Removal

What it does: Runs Independent Component Analysis (ICA) to separate the EEG signal into independent components, classifies each using ICLabel (automatic classifier), saves diagnostic plots for your review, then applies your chosen removals.

When to run: Once per participant per task. Requires human review of diagnostic plots before proceeding.

Windows:

```
python scripts/03_ica.py P01 stroop
python scripts/03_ica.py P01 nback
```

Mac:

```
python3 scripts/03_ica.py P01 stroop
python3 scripts/03_ica.py P01 nback
```

Reviewing ICA components: Each component is shown with: - **Left panel:** scalp topography — where on the head this component is strongest - **Right panel:** time series — how this component changes over the recording

Eye blink components show frontal scalp distribution and sharp deflections every few seconds. Muscle artefacts show lateral scalp distributions with high-frequency bursts.

 Example ICA Diagnostic Plot *(In this example: **IC00** is a classic eye blink, **IC04** is a muscle artifact with a focal edge, and **IC01/IC03** show typical dipolar brain activity. Eye movements would appear similarly to eye blinks but asymmetric across the frontal channels)*.

The script will pause and prompt you in the terminal to enter your decision based on the ICLabel suggestions and your visual inspection:

```
============================================================
STEP 7: YOUR DECISION
============================================================

Open component figures in Finder to inspect:
  output/ica/P01_stroop_ica_components_00_05.png
  output/ica/P01_stroop_ica_components_06_11.png
  ...

ICLabel automatic suggestion: remove [0, 2, 4, 7, 21]

Reminder — what to look for:
  Frontal bilateral blob (Fp1/Fp2) = eye blink    -> remove
  Asymmetric frontal               = eye movement -> remove
  Focal edge (temporal/frontal)    = muscle       -> consider removing
  Regular rhythmic pulses          = heartbeat    -> remove
  Smooth dipolar gradient          = brain        -> keep
  When in doubt                                   -> keep

Enter the FINAL list of component numbers to remove (comma-separated),
or press Enter to accept the automatic ICLabel suggestion as-is,
or type 'none' to remove nothing:
```

Produces: Diagnostic plots in `output/ica/` + `data/processed/P01_stroop_ica_raw.fif`

`04_epochs.py` — Epoching

What it does: Cuts the continuous EEG into short segments (epochs) time-locked to each stimulus onset. Applies baseline correction and rejects epochs exceeding ±75 µV.

Windows:

```
python scripts/04_epochs.py P01 stroop
python scripts/04_epochs.py P01 nback
```

Mac:

```
python3 scripts/04_epochs.py P01 stroop
python3 scripts/04_epochs.py P01 nback
```

Epoch parameters: Window −200 to +800 ms, baseline −200 to 0 ms, rejection ±75 µV.

Stroop trigger codes: S3 = stimulus, S5 = incongruent correct, S6 = congruent correct, S7 = no response

N-back trigger codes: S2 = stimulus, S7 = non-target correct, S8 = target hit, S9 = target miss

Produces: `data/processed/P01_stroop_epo.fif` + `output/epochs/P01_stroop_epoch_report.csv`

`05_erp.py` (Stroop) — Stroop ERP Extraction (per participant)

What it does: Averages epochs within each condition to produce individual ERP waveforms. Extracts amplitude and latency values for each ERP component. Saves waveform plots, topographic maps, and a CSV of component values. All participants are included; outlier flags recorded in CSV.

Windows:

```
python scripts/05_erp.py --task stroop P01
```

Mac:

```
python3 scripts/05_erp.py --task stroop P01
```

> 💡 **Tip: Stop here and use the pipeline runner!** If you have completed the manual steps 01–04 (including ICA review) for **all** participants, you no longer need to run `05_erp.py` or any subsequent scripts manually.
>
> Jump straight to **Section 9** and use `run_pipeline.py --task stroop` to automatically process all participants, generate the group plots, and run all statistical tests in one go!

Components extracted:

Component	Window	Electrode	Electrode location and neural source	Cognitive process	Type
N200	200–350 ms	FC1+FC2 (virtual FCz)	Frontocentral scalp, overlying anterior cingulate cortex (BA 24/32) and dorsolateral prefrontal cortex (BA 9/46)	Conflict monitoring between ink colour and word meaning; response selection	Primary
P300	300–600 ms	Pz	Parietal midline, overlying temporo-parietal junction (BA 39/40) and hippocampus	Context updating — the brain finalises its decision and updates working memory	Primary
P3b	300–600 ms	Pz	Same as P300	Peak latency only — compared with P300 latency to assess whether P3b drives the latency difference	Primary (latency only)
N1	80–160 ms	FC1+FC2 (virtual FCz)	Frontocentral, overlying frontal eye fields (BA 6/8) and supplementary motor area	Early attentional gating — how much resource the brain directed to the stimulus at onset	Exploratory

Component	Window	Electrode	Electrode location and neural source	Cognitive process	Type
CSW	400–700 ms	FC1+FC2 (virtual FCz)	Frontocentral, overlying ACC and lateral PFC	Conflict slow wave — sustained frontal engagement during incongruent trial resolution	Exploratory

Produces (per participant): - `output/erp/P01_stroop_butterfly.png` - `output/erp/P01_stroop_erp_FC1_FC2.png` — N1 (purple), N200 (blue), CSW (teal) windows - `output/erp/P01_stroop_erp_Pz.png` — P300/P3b window (red) - `output/erp/P01_stroop_erp_Fz.png` — frontal midline display only - `output/erp/P01_stroop_topomaps_combined.png` - `output/erp/P01_stroop_erp_components.csv`

`05_erp.py` (N-back) — N-back ERP Extraction (per participant)

What it does: Same as `05_erp.py` (Stroop) but for the N-back task with task-appropriate components.

Windows:

```
python scripts/05_erp.py --task nback P01
```

Mac:

```
python3 scripts/05_erp.py --task nback P01
```

> 💡 **Tip: Stop here and use the pipeline runner!** If you have completed the manual steps 01–04 (including ICA review) for **all** participants, jump straight to **Section 9** and run `python3 run_pipeline.py --task nback` to automatically finish the rest of the analysis!

Components extracted:

Component	Window	Electrode	Electrode location and neural source	Cognitive process	Type
N200	200–350 ms	FC1+FC2 (virtual FCz)	Frontocentral, overlying anterior cingulate cortex (BA 24/32) and dorsolateral PFC (BA 9/46)	Working memory updating — matching current letter against memory buffer and preparing response	Primary
P300	300–600 ms	Pz	Parietal midline, overlying temporo-parietal junction (BA 39/40) and hippocampus	Context updating — decision finalised, WM buffer updated	Primary
P3b	300–600 ms	Pz	Same as P300	Peak latency only	Primary (latency only)
N1	80–160 ms	FC1+FC2 (virtual FCz)	Frontocentral, overlying frontal eye fields (BA 6/8)	Early attentional gating at stimulus onset	Exploratory
P2	150–250 ms	Pz	Parietal midline, overlying superior parietal lobule (BA 7) and fusiform gyrus (BA 37)	Early stimulus classification — letter identity recognised, comparison against WM template begins	Exploratory
FSW	200–500 ms	Fz	Frontal midline, overlying medial PFC (BA 9/10) and anterior cingulate cortex (BA 24/32)	Frontal slow wave — sustained WM maintenance load; active holding of information in WM buffer	Exploratory

Produces (per participant): - `output/erp/P01_nback_butterfly.png` - `output/erp/P01_nback_erp_FC1_FC2.png` — N1 (purple) and N200 (blue) windows - `output/erp/P01_nback_erp_Pz.png` — P2 (orange) and P300/P3b (red) windows - `output/erp/P01_nback_erp_Fz.png` — frontal slow wave, display only - `output/erp/P01_nback_topomaps_combined.png` - `output/erp/P01_nback_erp_components.csv`

`06_group_erp.py` (Stroop) — Stroop Group ERP Analysis

What it does: Loads all individual ERP component CSVs, computes group grand averages, generates group-level waveform plots with individual participant traces behind the group average, produces bar charts comparing the two groups with p-value annotations.

Windows:

```
python scripts/06_group_erp.py --task stroop
```

Mac:

```
python3 scripts/06_group_erp.py --task stroop
```

Produces (see Section 10 for full list): Waveform plots, amplitude and latency bar charts, conflict effect plot, P3b vs P300 latency scatter, summary CSVs.

`06_group_erp.py` (N-back) — N-back Group ERP Analysis

Windows:

```
python scripts/06_group_erp.py --task nback
```

Mac:

```
python3 scripts/06_group_erp.py --task nback
```

`07_statistics.py` (Stroop) — Stroop Statistical Analysis

What it does: Between-group statistical comparisons for all primary ERP components and behavioural measures. Produces three output files: primary statistics (full tests), exploratory statistics (effect sizes only), sensitivity analysis (with and without outlier participants).

Windows:

```
python scripts/07_statistics.py --task stroop
```

Mac:

```
python3 scripts/07_statistics.py --task stroop
```

Statistical tests: - Welch t-test (does not assume equal variance) - Mann-Whitney U (non-parametric, more appropriate for small samples) - Cohen's d effect size (small < 0.5, medium 0.5–0.8, large > 0.8)

Significance notation: `*** p<.001` `** p<.01` `* p<.05` `† p<.10` `ns`

`07_statistics.py` (N-back) — N-back Statistical Analysis

Windows:

```
python scripts/07_statistics.py --task nback
```

Mac:

```
python3 scripts/07_statistics.py --task nback
```

---

9. Running the Pipeline

Recommended — use the pipeline runners

> ⚠️ **CRITICAL TIMING:** Do not run this script until you have manually completed Steps 01 through 04 for all participants! Because `03_ica.py` requires human review, the first half of the pipeline cannot be fully automated.
>
> Once every participant has an epoched file (`*_epo.fif`), `run_pipeline.py` will automatically execute the rest of the workflow: ERP extraction, group averaging, statistics, and behavioural analysis.

Windows:

```
cd C:\Users\yourname\eeg_analysis
.venv\Scripts\activate
```

```
python run_pipeline.py --task stroop
python run_pipeline.py --task nback
```

Mac:

```
cd /Users/yourname/eeg_analysis
source .venv/bin/activate
python3 run_pipeline.py --task stroop
python3 run_pipeline.py --task nback
```

Single participant only:

Windows:

```
python run_pipeline.py --task stroop P01
python run_pipeline.py --task nback P01 P02 P04
```

Mac:

```
python3 run_pipeline.py --task stroop P01
python3 run_pipeline.py --task nback P01 P02 P04
```

Manual step-by-step order

```
# Step 1 — behavioural data (run once)

  Windows:  python  scripts/01_parse_eprime.py --task nback
  Mac:      python3 scripts/01_parse_eprime.py --task nback

  Windows:  python  scripts/01_parse_eprime.py --task stroop
  Mac:      python3 scripts/01_parse_eprime.py --task stroop

# Step 2 — EEG import and filter (per participant per task)

  Windows:  python  scripts/02_import_filter_eeg.py P01 stroop JD_Stroop_22_05_
  Mac:      python3 scripts/02_import_filter_eeg.py P01 stroop JD_Stroop_22_05_

  Windows:  python  scripts/02_import_filter_eeg.py P01 nback  JD_Nback_22_05_2
  Mac:      python3 scripts/02_import_filter_eeg.py P01 nback  JD_Nback_22_05_2

# Step 3 — ICA (per participant per task)
# → Review ICA plots in output/erp/ before proceeding
```

```
  Windows:  python  scripts/03_ica.py P01 stroop
  Mac:      python3 scripts/03_ica.py P01 stroop

# Step 4 — Epoching (per participant per task)

  Windows:  python  scripts/04_epochs.py P01 stroop
  Mac:      python3 scripts/04_epochs.py P01 stroop

# Step 5 — Individual ERP (per participant, run both tasks)

  Windows:  python  scripts/05_erp.py --task stroop P01
  Mac:      python3 scripts/05_erp.py --task stroop P01

  Windows:  python  scripts/05_erp.py --task nback P01
  Mac:      python3 scripts/05_erp.py --task nback P01

# Step 6 — Group ERP (once, after all participants complete)

  Windows:  python  scripts/06_group_erp.py --task stroop
  Mac:      python3 scripts/06_group_erp.py --task stroop

  Windows:  python  scripts/06_group_erp.py --task nback
  Mac:      python3 scripts/06_group_erp.py --task nback

# Step 7 — Statistics (once)

  Windows:  python  scripts/07_statistics.py --task stroop
  Mac:      python3 scripts/07_statistics.py --task stroop

  Windows:  python  scripts/07_statistics.py --task nback
  Mac:      python3 scripts/07_statistics.py --task nback
```

> **Important:** Script 04 (ICA) requires human review of component plots before continuing. The pipeline runner will pause and prompt you to review the diagnostic images saved to `output/erp/` before applying removals.

---

10. Output Files — What You Get

Individual ERP outputs (`output/erp/`)

File	What it shows
`P01_stroop_butterfly.png`	All 32 channels overlaid for each condition. Key electrodes (FC1+FC2, Pz) shown in bold.

File	What it shows
	Useful for checking overall data quality and identifying noise.
`P01_stroop_erp_FC1_FC2.png`	ERP waveform at virtual FCz (average of FC1 and FC2). Three colour-coded time windows: N1 (purple), N200 (blue), CSW (teal).
`P01_stroop_erp_Pz.png`	ERP waveform at Pz. P300/P3b window shaded in red.
`P01_stroop_erp_Fz.png`	ERP waveform at Fz — display only. Shows frontal slow wave.
`P01_stroop_topomaps_combined.png`	Scalp maps at 5 time points showing voltage distribution across the scalp.
`P01_stroop_erp_components.csv`	Numerical component values: mean amplitude, peak amplitude, peak latency per component per condition. Includes `outlier_flag` column.

Group ERP outputs (`output/group/`)

File	What it shows
`group_stroop_erp_FC1_FC2.png`	Grand average waveforms — Group 1 (left panel), Group 2 (right panel). Bold line = group average. Thin semi-transparent lines = individual participants.
`group_stroop_erp_FC1_FC2_by_condition.png`	One panel per condition (congruent, incongruent, no response). Both groups overlaid: solid = Group 2, dashed = Group 1.
`group_stroop_N200_amplitude_bars.png`	Bar chart: Group 1 (left bar) vs Group 2 (right bar) per condition. Error bars = SEM. Dots = individual

File	What it shows
	participants. Significance bracket with p-value stars above each pair.
`group_stroop_N200_latency_bars.png`	Same layout as amplitude bars but for peak latency in milliseconds.
`group_stroop_P3b_vs_P300_latency.png`	Scatter plot: each point is one participant. X = P300 peak latency, Y = P3b peak latency. Points on the diagonal mean both measures found the same peak.
`group_stroop_conflict_effect.png`	Incongruent – congruent amplitude difference for N200 and P300 with p-value annotations.
`group_stroop_summary.csv`	Group means, SDs, SEMs per component per condition per group.
`group_stroop_individual_components.csv`	All individual participant values — used by script 07 for statistics.

Statistics outputs (`output/stats/`)

File	What it contains
`stroop_statistics_primary.csv`	Full results for primary outcomes: group means, SDs, Welch t-statistic, t-test p-value, significance stars, Mann-Whitney U, U p-value, Cohen's d, effect size interpretation, outlier participants flagged.
`stroop_statistics_exploratory.csv`	Cohen's d only for exploratory components. No p-values — exploratory observations only.
`stroop_statistics_sensitivity.csv`	Primary statistics re-run without flagged outlier participants. Compare with primary results to assess robustness.

Epoch reports (`output/epochs/`)

File	What it contains
`P01_stroop_epoch_report.csv`	Trial counts per condition: before rejection, after rejection, number removed, percentage removed, threshold used.

---

11. ERP Components Reference

What is an ERP component?

An ERP component is a characteristic peak or trough in the averaged EEG waveform that occurs at a predictable time after a stimulus and reflects a specific cognitive process. Components are named by their polarity (N = negative, P = positive) and approximate peak latency in milliseconds (e.g. N200 = negative peak around 200 ms).

Components measured in this pipeline

N1 (80–160 ms) — Exploratory

Electrode: FC1+FC2 (virtual FCz) — frontocentral midline
Brain source: Frontal eye fields (Brodmann area 6/8), supplementary motor area (BA 6)
Cognitive meaning: Early attentional gating — the earliest cortical sign that the brain directed attention to the stimulus at the moment it appeared. Larger N1 = more attentional resource allocated per stimulus.

P2 (150–250 ms) — Exploratory, N-back only

Electrode: Pz — parietal midline
Brain source: Superior parietal lobule (BA 7), fusiform gyrus (BA 37)
Cognitive meaning: Early stimulus classification — the brain identifies the letter and begins matching it against stored representations. Reflects the boundary between sensory processing and cognitive processing.

N200 (200–350 ms) — Primary

Electrode: FC1+FC2 (virtual FCz) — frontocentral midline
Brain source: Anterior cingulate cortex (BA 24/32), dorsolateral prefrontal cortex (BA 9/46)

Cognitive meaning: - *Stroop:* Conflict monitoring — the brain detects competition between the written word and the ink colour - *N-back:* Working memory updating — the brain matches the current letter against the memory buffer

P300 (300–600 ms) — Primary

Electrode: Pz — parietal midline
Brain source: Temporo-parietal junction (BA 39/40), hippocampus, posterior cingulate cortex (BA 23/31)
Cognitive meaning: Context updating — the brain finalises its decision, updates working memory, and consolidates the response. P300 latency is a particularly sensitive index of cognitive processing speed.

P3b (300–600 ms) — Primary (latency only)

Electrode: Pz — parietal midline
Brain source: Same as P300
Cognitive meaning: Named separately from P300 to allow comparison of the P3b-specific peak latency against the broader P300 window peak latency. If both measures agree for a participant, the P300 latency difference is driven by the canonical P3b.

Conflict Slow Wave / CSW (400–700 ms) — Exploratory, Stroop only

Electrode: FC1+FC2 — frontocentral
Brain source: Anterior cingulate cortex, lateral prefrontal cortex
Cognitive meaning: Sustained conflict resolution — prolonged frontal engagement during resolution of incongruent Stroop trials. Often visible at the grand average level depending on group condition.

Frontal Slow Wave / FSW (200–500 ms) — Exploratory, N-back only

Electrode: Fz — frontal midline
Brain source: Medial prefrontal cortex (BA 9/10), anterior cingulate cortex (BA 24/32)
Cognitive meaning: Active working memory maintenance — the frontal cortex sustaining attention and holding information in the WM buffer. Larger and more sustained FSW = deeper WM engagement per stimulus.

Representative Traces

Below are the group-average ERP traces showing these components extracted directly from this pipeline:

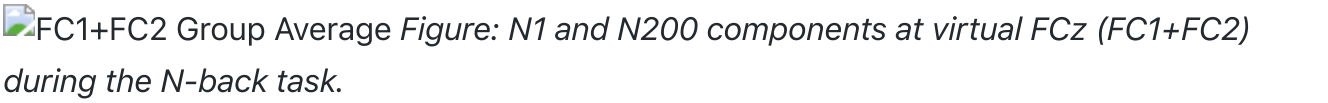

Figure: N1 and N200 components at virtual FCz (FC1+FC2) during the N-back task.

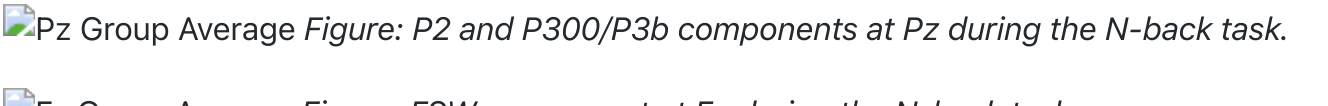

Figure: P2 and P300/P3b components at Pz during the N-back task.

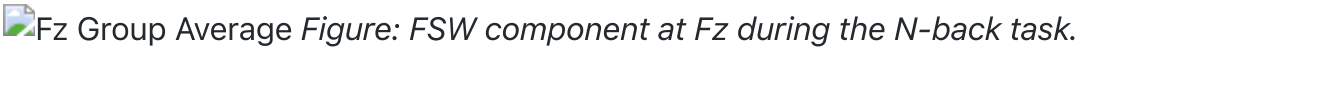

Figure: FSW component at Fz during the N-back task.

ERP plot conventions

Convention	Meaning
Negative up	Standard ERP convention — negative values shown at top of plot
Green	Congruent correct (Stroop) / Non-target correct (N-back)
Red	Incongruent correct (Stroop) / Target miss (N-back)
Blue	Target hit (N-back)
Grey	No response (Stroop)
Solid line	Group 2
Dashed line	Control group
Bold line	Group grand average
Thin faint lines	Individual participants
Shaded regions	Component measurement windows
Error bars	Standard error of the mean (SEM)
Stars on bar charts	`*** p<.001` `** p<.01` `* p<.05` `† p<.10` `ns`

---

12. Troubleshooting

"Module not found" error

```
ModuleNotFoundError: No module named 'mne'
```

Fix: Check the virtual environment is activated (you should see `(.venv)` in your prompt), then:

Windows: `python -m pip install mne numpy scipy pandas matplotlib`
Mac: `python3 -m pip install mne numpy scipy pandas matplotlib`

"File not found" for .vhdr

Fix: Check the `.vhdr`, `.vmrk`, and `.eeg` files are all in `data/raw/` and the filename you typed matches exactly (case-sensitive on Mac).

"python not recognised" on Windows

The Python installer was run without ticking "Add Python to PATH".
Fix: Uninstall Python and reinstall, making sure to tick the PATH checkbox on the first screen.

"python3 not found" on Mac

Fix: Open Terminal and type `python3 --version`. If not found, install Python from python.org.

ICA fails with montage error

Fix: Re-run script 02. Make sure the correct 32-channel recording file is being used.

Very few epochs after rejection

If fewer than 20 trials per condition, ERP averages will be unstable. This participant is flagged in the output CSV with an `outlier_flag` value. They are included in the analysis but noted separately in the sensitivity analysis.

"Permission denied" on Windows

Fix: Run Command Prompt as Administrator, or move the project to `C:\Users\YourName\`.

Colours not showing in terminal on Windows

Fix: Use Windows Terminal (free from Microsoft Store) or PowerShell instead of the older Command Prompt.

---

13. Study Design Reference

Participants

- Any number of participants split across two groups
- Between-subjects design

EEG recording

- System: BrainVision actiCHamp, 32 channels, 500 Hz
- Online reference: Cz (re-added in script 02)
- Analysis reference: average reference
- Filter: 0.1–40 Hz bandpass + 50/100 Hz notch

Statistical approach

- Primary: Welch t-test + Mann-Whitney U, both reported
- Effect size: Cohen's d (primary metric given small N)
- Primary outcomes: full p-values
- Exploratory outcomes: Cohen's d only
- All participants included; outlier flags in CSV
- Sensitivity analysis re-runs without flagged participants

Key references

- Kappenman, E.S. & Luck, S.J. (Eds.) (2012). *The Oxford Handbook of Event-Related Potential Components*. Oxford University Press.
- Folstein, J.R. & Van Petten, C. (2008). Influence of cognitive control and mismatch on the N2 component of the ERP: A review. *Psychophysiology*, 45(1), 152–170. https://doi.org/10.1111/j.1469-8986.2007.00602.x
- Polich, J. (2007). Updating P300: An integrative theory of P3a and P3b. *Clinical Neurophysiology*, 118(10), 2128–2148. https://doi.org/10.1016/j.clinph.2007.04.019
- Jonides, J., Schumacher, E.H., Smith, E.E., Lauber, E.J., Awh, E., Minoshima, S. & Koeppe, R.A. (1997). Verbal working memory load affects regional brain activation as measured by PET. *Journal of Cognitive Neuroscience*, 9(4), 462–475. https://doi.org/10.1162/jocn.1997.9.4.462
- Gevins, A. & Smith, M.E. (2000). Neurophysiological measures of working memory and individual differences in cognitive ability and cognitive style. *Cerebral Cortex*, 10(9), 829–839. https://doi.org/10.1093/cercor/10.9.829
- Hillyard, S.A. & Anllo-Vento, L. (1998). Event-related brain potentials in the study of visual selective attention. *Proceedings of the National Academy of Sciences*, 95(3), 781–787. https://doi.org/10.1073/pnas.95.3.781

- Kok, A. (2001). On the utility of P3 amplitude as a measure of processing capacity. *Psychophysiology*, 38(3), 557–577. https://doi.org/10.1017/S0048577201990559
